



Análise e Projeto de Sistemas

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO DE AULAS PRÁTICAS

Nome: Vitória Freitas

RA: 2523279

Polo de matrícula: Carapicuíba - Vila Dirce

Local da realização da Aula Prática: Alphaville - Santana de Parnaíba

Ano da postagem: 2025

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Introdução à Análise e Projeto de Sistemas	Relatório 1
--	---	--------------------

1. Resumo Teórico

O ciclo de vida do desenvolvimento é um conceito que envolve toda a parte da construção de um sistema, desde sua concepção até sua desativação. A compreensão deste ciclo é extremamente importante para eficiência e qualidade do software, possibilitando manutenção durante ele. O ciclo aborda todas as fases de vida do software, como: fase de planejamento, fase de desenvolvimento, fases de testes, fase de implantação e manutenção, por fim, a fase de desativação.

A metodologia ágil é uma metodologia de desenvolvimento de projetos, ela divide os projetos maiores em pequenas partes valorizando a flexibilidade e colaboração. O scrum é uma metodologia ágil que ajuda a equipe a entregar o produto de maneira incremental e colaborativa. O XP é uma metodologia ágil para equipes pequenas a médio tamanho, para desenvolvimento de software em que os requisitos podem mudar rapidamente.

A metodologia em cascata existe desde 1970, surgiu para solucionar problemas de grandes projetos de desenvolvimento de software. Dentro dessa metodologia os projetos são divididos em etapas que se sucedem umas das outras, seguindo um caminho descendente em um formato de cascata. Os processos dentro dessa metodologia incluem: coleta de requisitos, design do produto, implementação, testes e manutenção.

Para exemplificar, vou dar de exemplo a construção de um sistema web. Para o desenvolvimento de um sistema web utilizando uma metodologia ágil você poderia ter uma maior flexibilidade para mudanças de requisitos e se encaixaria melhor em uma equipe de médio a pequeno tamanho. Para desenvolvimento de grande sistema web com uma grande equipe a metodologia cascata, poderia se encaixar bem dentro do desenvolvimento de software.

2. Estudo de Caso

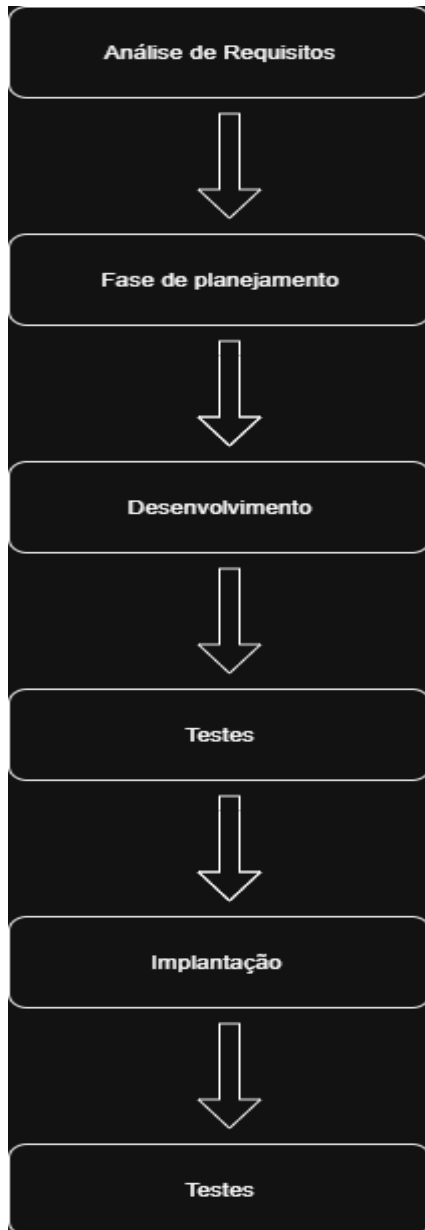
Um sistema de entrega para pedidos de produtos de um supermercado. Neste sistema, clientes podem pedir produtos que estejam disponíveis para serem entregues no local de entrega cadastrado dentro do aplicativo. Desenvolvido pensando na segurança de dados, com criptografia para os dados sensíveis.

As fases incluídas dentro do desenvolvimento desse software foram a análise de

requisitos, a fase de planejamento, desenvolvimento, testes, implantação e manutenção.

Pontos de benefícios das metodologias ágeis seriam a flexibilidade que elas trazem, caso ocorra alguma mudança durante o desenvolvimento do software elas ajudariam a equipe a se adaptar para essas mudanças. E podemos citar, a melhora na colaboração dentro das equipes ao utilizar uma metodologia ágil.

3. Diagrama do Ciclo de Vida



4. Reflexões Finais

Dificuldades para identificar o ciclo de vida, foram para saber quais ciclos são envolvidos dentro do desenvolvimento do software. Qual vai ser mais eficiente entre a metodologia ágil e a cascata vai depender do tipo de sistema você e sua equipe estará desenvolvendo, no tamanho dele, sua complexidade, tamanho e experiência de sua equipe e outros fatores.

Aprendi mais sobre como acontece o desenvolvimento de software na prática, sobre

metodologias que são utilizadas e sobre sua eficiência para o desenvolvimento e na melhora na colaboração da equipe.

5. Referências

<https://uds.com.br/blog/ciclo-de-vida-do-software-web/>

<https://monday.com/blog/pt/desenvolvimento/metodologia-agil-x-em-cascata-qual-tipo-de-gere-n-te-voce-e/>

<https://www.scrum.org/resources/what-scrum-module>

<https://www.devmedia.com.br/integrando-xp-as-principais-metodologias-ageis/30989>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Levantamento de Requisitos	Relatório 2
--	---	--------------------

1. Resumo Teórico

Antes de iniciar o desenvolvimento de um software é importante entender as necessidades do cliente, para isso podemos fazer o levantamento de requisitos. Os requisitos fazem a ponte entre o cliente e o programador. Suas principais características são: buscar informações do cliente para construção do software, organizar o projeto e entregar um software de qualidade.

Requisitos funcionais descrevem as ações que o software deve fazer, são as operações que o software deve executar. Por exemplo, o site deve fazer uma busca e filtragem dos dados. Requisitos não funcionais estão ligados a parte da performance do software, como por exemplo: desempenho, segurança, confiabilidade, usabilidade, manutenibilidade e portabilidade.

Entre as principais técnicas de elicitação de requisitos, podemos citar a entrevista, brainstorming e a prototipação. A entrevista é uma reunião com as pessoas envolvidas no projeto (stakeholder) sobre o sistema que será desenvolvido, com ela podemos fazer a elicitação de requisitos e entender as necessidades do cliente. O brainstorming é uma reunião ou várias para que as pessoas possam sugerir e explorar ideias. A prototipação inicia pelo estudo dos requisitos do usuário e um protótipo do sistema para ser validado pelo cliente, os requisitos iniciais servem para fazer um protótipo da interface do usuário, de maneira mais simplificada.

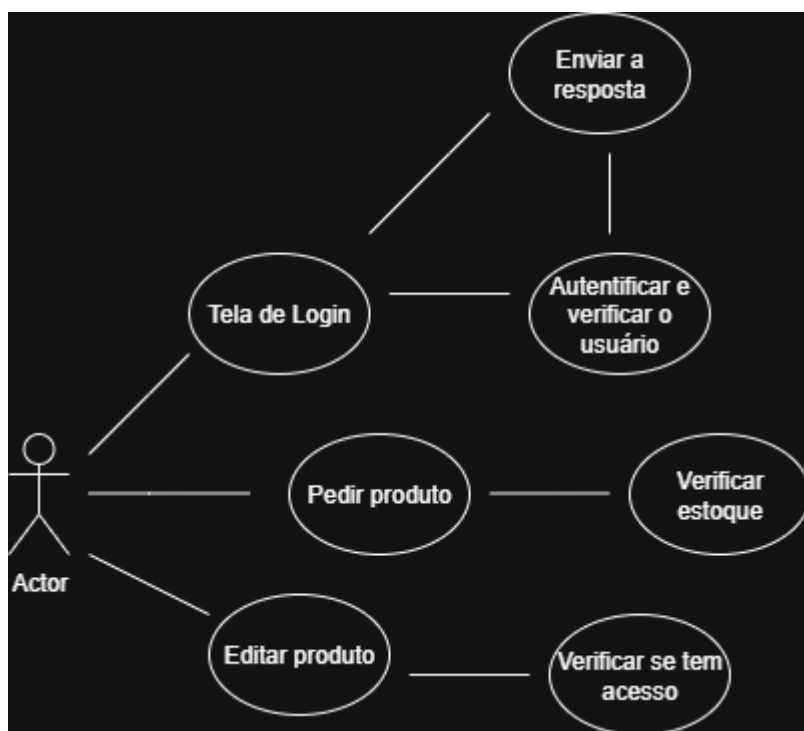
2. Lista de Requisitos

Número do requisitos	Requisitos Funcionais
RF01	O sistema deve permitir login de usuários
RF02	O sistema deve mostrar a quantidade de produtos no estoque.
RF03	O sistema deve ter edição de produtos no estoque.
RF04	O sistema deve ter adição de produtos no estoque.

RF05	O sistema deve ter uma área para usuários administradores e outra para os usuários que vão comprar os produtos.
-------------	---

Número do requisitos	Requisitos Não Funcionais
RNF01	O sistema deve responder em até 2 segundos.
RNF02	O sistema deve ter autenticação.
RNF03	O sistema deve ter criptografia.
RNF04	O sistema deve ter uma interface intuitiva.
RNF05	O sistema deve ter um firewall para proteção contra acessos não autorizados.

3. Diagrama de Caso de Uso



4. Reflexões Finais

Na minha perspectiva as maiores dificuldades em levantar requisitos é entender as necessidades do usuário e saber se ela é funcional ou não funcional. Na minha opinião, a técnica mais eficiente é a de entrevista, pois com ela podemos saber quais são as expectativas e necessidades do cliente. Com essa aula, aprendi melhor sobre o processo de desenvolvimento de software.

5. Referências

<https://www.devmedia.com.br/levantamento-de-requisitos/>

<https://querobolsa.com.br/revista/requisitos-funcionais-e-nao-funcionais>

<https://acertbr.com.br/tecnicas-de-elicitacao-de-requisitos/>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Modelagem de Sistemas com UML	Relatório 3
--	--	--------------------

1. Resumo Teórico

A Linguagem de Modelagem Unificada (UML), é uma linguagem gráfica padronizada que tem a função de descrever, projetar e visualizar para documentar sistemas orientados a objetos. Ela é importante, pois auxilia os desenvolvedores com visualização, compreensão, projeção do sistema, identificar padrões, documentação e na detecção de problemas. Ela facilita a comunicação durante o desenvolvimento de software.

2. Sistema Modelado

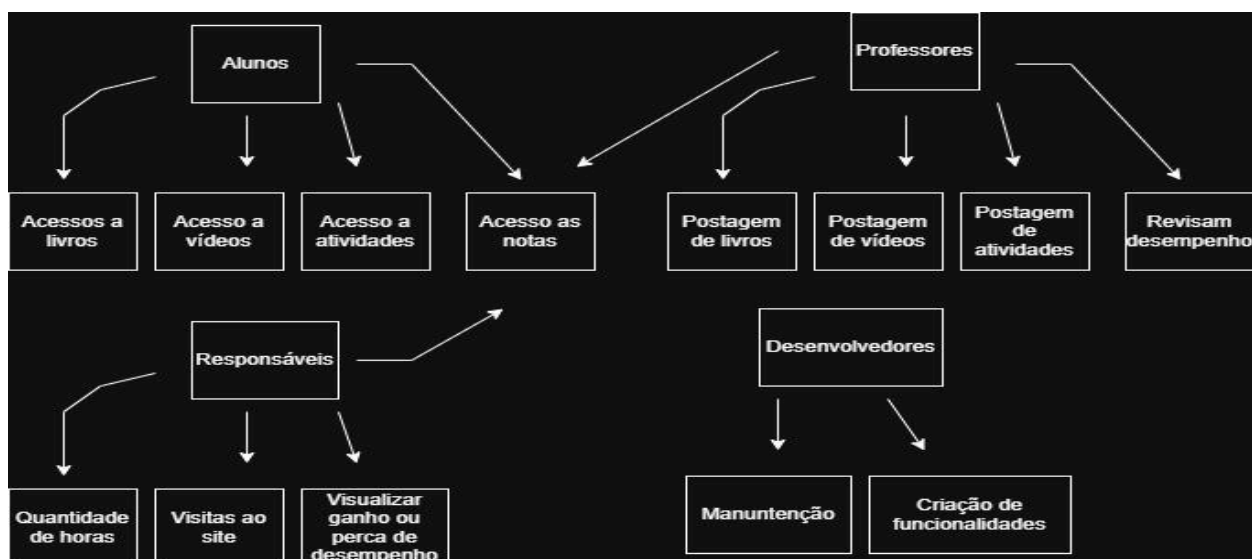
Uma plataforma de ensino educacional para crianças e adolescentes, com o objetivo de conectar alunos, professores e responsáveis.

Atores do sistema:

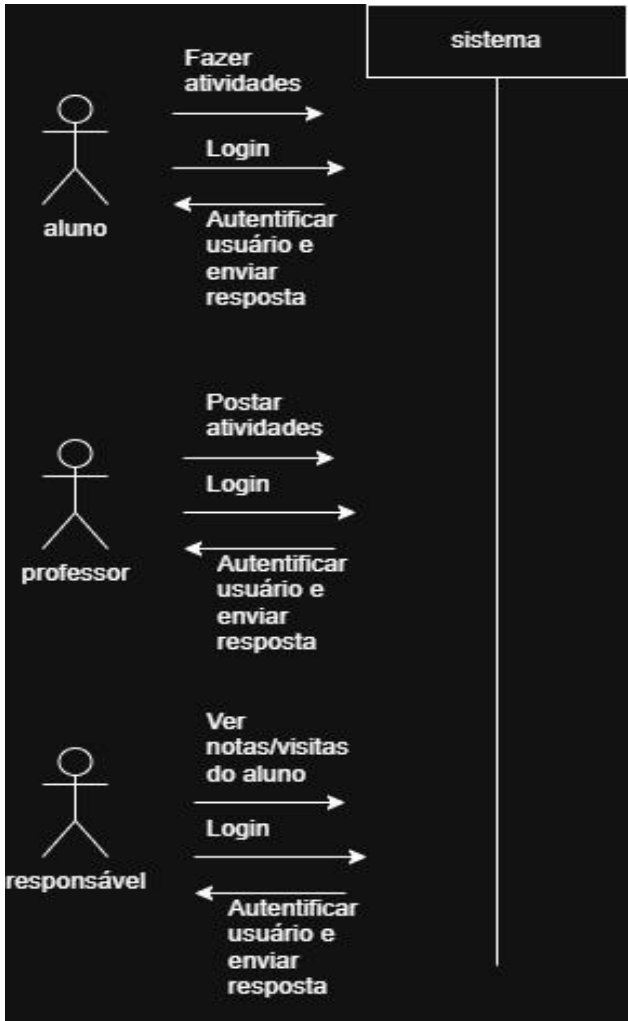
1. Professores: postam atividades, revisam desempenho dos alunos, postam vídeo aulas e livros complementares.
2. Alunos: têm acessos às atividades e livros, aulas complementares.
3. Responsáveis: têm acesso às notas dos alunos, visitas que fazem ao site, quantidades de horas e podem visualizar, se houve melhora ou piora no desempenho do aluno.
4. Desenvolvedores: responsáveis pela criação e manutenção do site.

3. Diagramas Criados

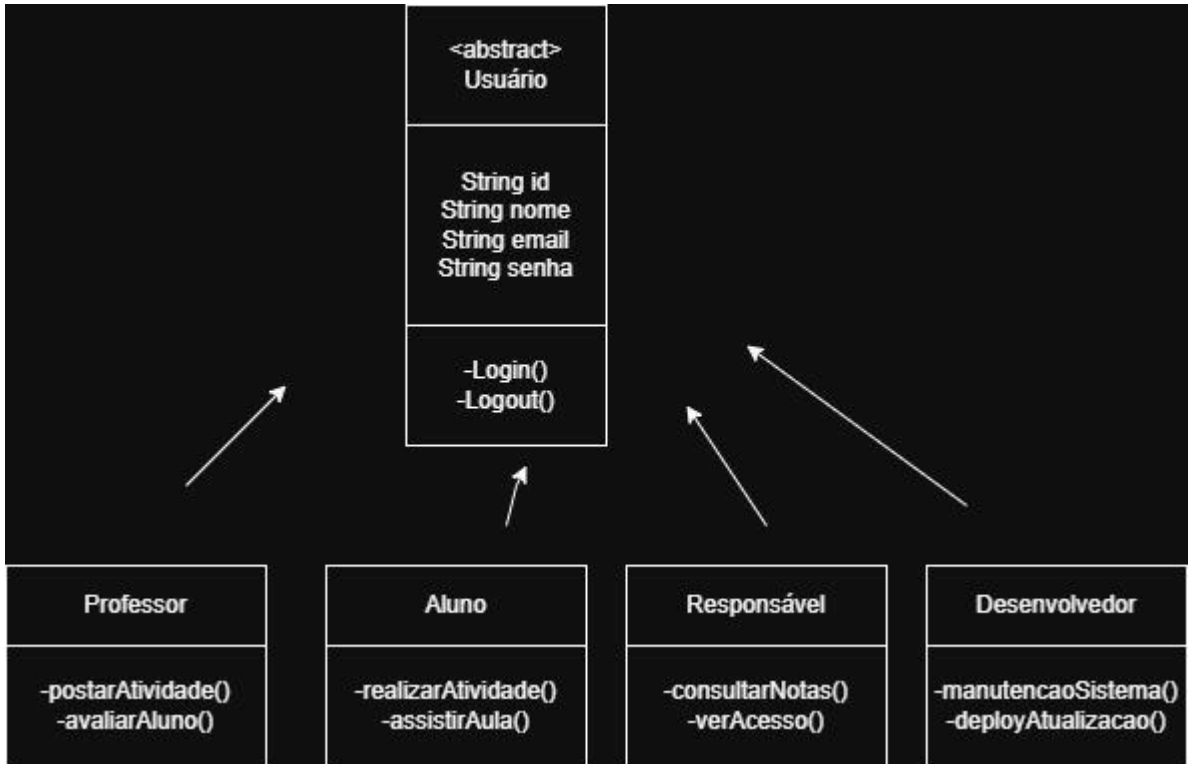
Caso de uso:



Sequência:



Classes:



4. Reflexões Finais

Dificuldade em relação a criação dos diagramas foi entender qual é a proposta de cada um e como encaixar com as necessidades do sistemas. Os diagramas são importantes, pois com eles temos um melhor entendimento sobre o sistema que temos que criar e como ele deve funcionar. Aprendemos na prática como devemos fazer esses diagramas.

5. Referências

<https://www.dio.me/articles/a-importancia-da-uml-na-engenharia-de-software-simplificando-a-complexidade-do-desenvolvimento>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Design de Software	Relatório 4
--	---	--------------------

1. Resumo Teórico

Padrões de design são soluções genéricas para problemas comuns nos códigos de desenvolvimento do software. Eles são um conceito geral para ajudar a resolver problemas recorrentes. São usados, porque facilitam o trabalho dos desenvolvedores e aumentam a produtividade, permitindo a eles criar códigos melhores e de maneira mais rápida. Exemplos de Padrões de design: Singleton, classes tem apenas uma instância; Factory, cria objetos em especificar a classe; Observer, define dependência para muitos objetos. A arquitetura multicamadas tem um princípio básico que o cliente não se comunica diretamente com o servidor de banco de dados e sim intermediária (como o banco de dados), essa arquitetura utiliza objetos distribuídos e interfaces para realizar suas operações. A arquitetura multicamadas e os padrões de design são elementos complementares, que deixam o desenvolvimento mais eficiente.

2. Código Desenvolvido

```
class ConnectionManager:
    _instance = None
    _connections = {}

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            print("ConnectionManager criado pela primeira vez")
        return cls._instance

    def __init__(self):
        # O __init__ será chamado toda vez, mas a instância é a mesma
        print("ConnectionManager inicializado")

    def add_connection(self, name, connection):
        self._connections[name] = connection
        print(f"Conexão '{name}' adicionada")

    def get_connection(self, name):
```

```

        return self._connections.get(name)

    def close_all(self):
        for name in self._connections:
            print(f"Fechando conexão: {name}")
        self._connections.clear()

manager1 = ConnectionManager()
manager1.add_connection("database", "mysql_connection")

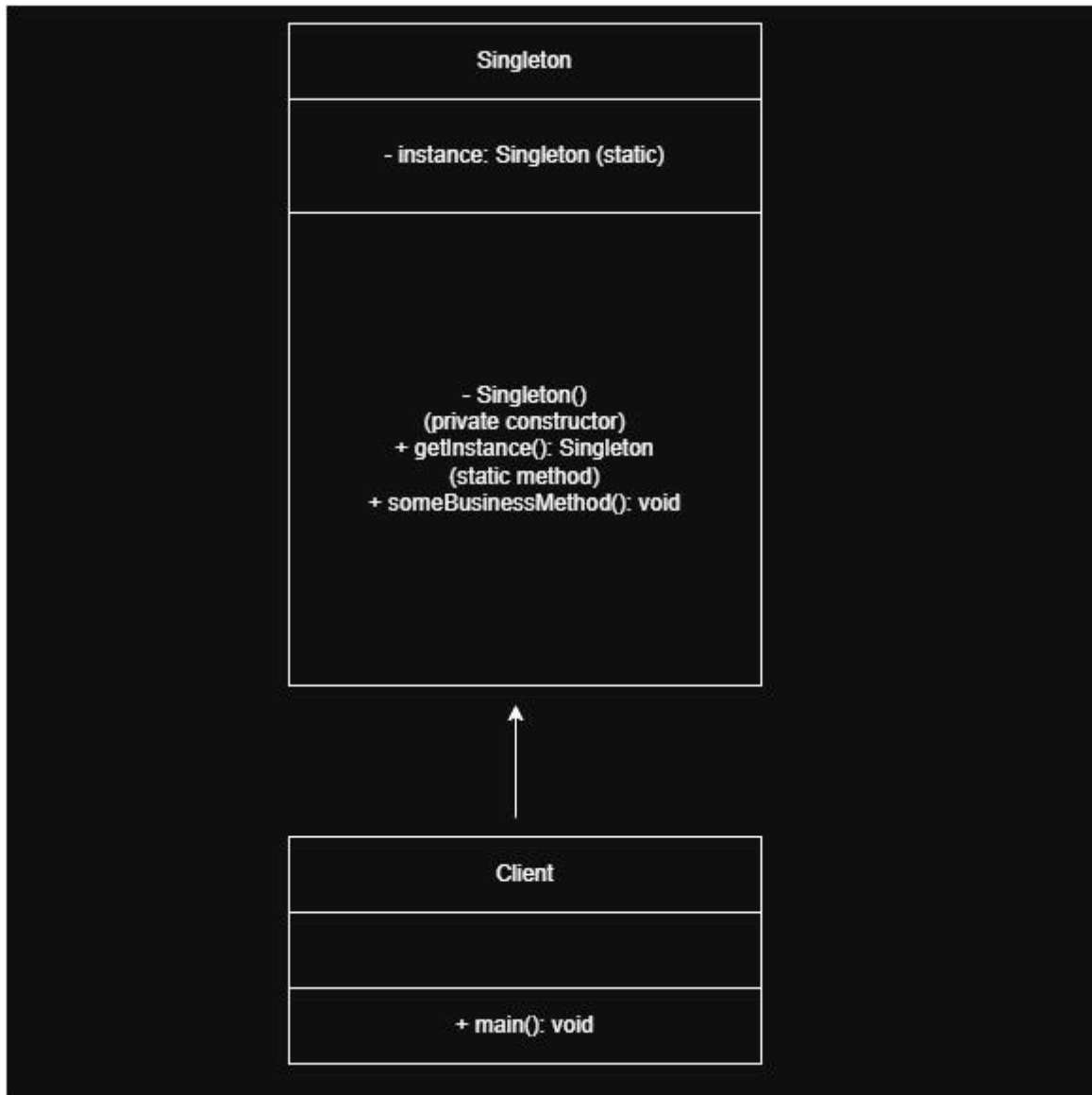
# manager2 NÃO é uma nova instância, é apenas outra referência à MESMA
# instância
manager2 = ConnectionManager()
manager2.add_connection("api", "rest_api_connection")

print(f"manager1 é manager2? {manager1 is manager2}") # True
print(f"Conexões no manager1: {manager1._connections}")
print(f"Conexões no manager2: {manager2._connections}")

# Todas as referências apontam para o mesmo objeto
manager3 = ConnectionManager()
print(f"Conexões no manager3: {manager3._connections}")

```

3. Diagrama UML



4. Reflexões Finais

Dificuldades para implementar foi entender como cada padrão de design funciona e como implementá-las dentro do código. Esses padrões ajudam a solucionar problemas recorrentes dentro dos códigos. Com essa aula aprendi na prática como implementar esses padrões dentro de projetos.

5. Referências

<https://uds.com.br/blog/a-importancia-dos-design-patterns-no-dev-de-software/>

<https://www.devmedia.com.br/introducao-ao-modelo-multicamadas/5541>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Arquitetura de Software	Relatório 5
--	--	--------------------

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Ferramentas de Modelagem e Case	Relatório 6
--	--	--------------------

1. Resumo Teórico

(Escreva aqui um resumo de 8 a 10 linhas abordando:)

- O que são ferramentas CASE.
- Exemplos de ferramentas (Astarh, Visual Paradigm, Draw.io).
- Benefícios do uso dessas ferramentas no desenvolvimento de sistemas.

2. Diagramas Criados

(Inserir as capturas de tela dos diagramas de caso de uso, classes e sequência gerados na ferramenta.)

3. Descrição do Processo

Descreva o processo de modelagem: quais recursos da ferramenta foram utilizados e quais foram as dificuldades.

4. Reflexões Finais

- Quais foram os maiores benefícios ao usar a ferramenta?
- Como as ferramentas CASE auxiliam na documentação e no desenvolvimento?
- O que você aprendeu nesta prática?

5. Referências

(Inclua pelo menos 1 referência do livro-texto e outras fontes utilizadas.)

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Avaliação e Validação de Projetos de Sistemas	Relatório 7
--	---	--------------------

1. Resumo Teórico

(Escreva aqui um resumo de 8 a 10 linhas abordando:)

- A importância da avaliação e validação em projetos de sistemas.
- Diferença entre verificação e validação.
- Exemplos de métodos de revisão colaborativa.

2. Checklist de Avaliação

Inclua o checklist utilizado para avaliar os diagramas ou documentos de outro grupo.

Exemplo de itens:

- Diagramas estão completos e coerentes?
- Requisitos estão claros e documentados?
- Há problemas de consistência ou duplicação?

3. Resultados da Avaliação

Descreva os problemas encontrados, as correções sugeridas e os pontos fortes do projeto avaliado.

4. Reflexões Finais

- Quais foram os principais aprendizados durante a avaliação?
- Como o feedback em grupo contribui para a melhoria do projeto?
- O que você aprendeu nesta prática?

5. Referências

(Inclua pelo menos 1 referência do livro-texto e outras fontes utilizadas.)

Instituto de Ciências Exatas e Tecnologia	Disciplina: Análise e Projeto de Sistemas Título da Aula: Manutenção e Evolução de Sistemas	Relatório 8
--	--	--------------------

1. Resumo Teórico

(Escreva aqui um resumo de 8 a 10 linhas abordando:)

- Tipos de manutenção de software (corretiva, adaptativa, perfectiva e preventiva).
- Importância da refatoração e da reengenharia.
- Exemplos práticos de evolução de sistemas.

2. Código Original

(Cole aqui o código-fonte original antes da refatoração.)

3. Código Refatorado

(Cole aqui o código-fonte atualizado após a refatoração, destacando melhorias aplicadas.)

4. Reflexões Finais

- Quais foram as dificuldades ao realizar a manutenção?
- Que melhorias foram alcançadas?
- O que você aprendeu nesta prática?

5. Referências

(Inclua pelo menos 1 referência do livro-texto e outras fontes utilizadas.)